Month 1: Week 4



Bash Scripting, Automation & Cron

Bash Scripting

First off, what is Bash?

Bourne Again Shell- Bash is a default shell in many Linux systems. Like we have other types of shells, sh(the original Unix shell), Zsh (bash compatible but with more features...default shell on Mac, Fish (Friendly Interactive shell; has autosuggestions, very beginner-friendly), Ksh (KornShell), Powershell (Microsoft's shell for Windows), Dash (lightweight shell used in Debian-based systems for scripts), Ash (Embedded systems).

So, if we say Bash Scripting, what does it mean?

Bash scripting is the practice of writing a series of commands for the Bash Shell (Bourne Again Shell), so they can be executed automatically rather than by typing them one by one. A Bash script is just a plain text file containing a series of commands that the Bash shell can execute.

Think of it like this, instead of running 10 terminal commands manually every time, you can put them in a single .sh file, run that file once, and Bash executes them in order.

How would you feel about clicking through 500 files to rename them one by one?" That's what life without scripting looks like.

How does Scripting help in DevOps and Cloud Engineering

- It helps automate repetitive tasks.
- Deploying Applications
- Automating Server Setup
- Processing log files (Log levels: WARN, DEBUG, TRACE, INFO, CRITICAL, ERROR)
- Backup scripts
- Running CI/CD pipeline steps



Rules for Writing Bash Scripts

- Start with the shebang (#!/bin/bash), which must be on the first line.
- Comments start with #, they are ignored by Bash except for the shebang.
- Commands are run in order, top to bottom, just like in the terminal.
- Variables

No spaces around = when assigning:

- ❑ name="John" # ✅ correct
- ❑ name = "John" # ❌ wrong

Use $variable to access it:

- ❑ echo $name
- Commands must be valid shell commands (Same as typing in terminal.)

Bash Script Hands-On: Steps to Write a Bash File

Step 1: Create a file

Step 2: Add the shebang (VERY important): This tells the system to run the file with Bash, without it, the system might try to run it with another shell or complain.

- #!/bin/bash

Step 3: Write your commands

- #!/bin/bash
- # This is a comment
- echo "Hello, World!"
- name="Mayowa"
- echo "Your name is $name"

Step 4: Make the script executable

- chmod +x myscript.sh

Step 5: Run the script; You can run it in two ways:

- ./myscript.sh # If you're in the same directory
- bash myscript.sh # Explicitly using bash

Check out file extensions.

- .yaml
- .css
- .html
- .js
- .json
- .md
- .xml
- .config
- .env
- .txt
- .sh
- Sql
- .log



An Example Script

```
#!/bin/bash

set -e # Exit immediately if a command fails

# Ask for a name

echo "What is your name?"

read name

# Greet the user

echo "Hello, $name!"

# Show current date

echo "Today is $(date)"
```

Variables

Variables are used to store values you want to reuse.

Rules:

- No spaces around `=`
- Reference with `$` prefix
- Use quotes around strings with spaces

➔ #!/bin/bash
➔ NAME="Mayowa"
➔ echo "Hello, $NAME"

User input with read:

➔ #!/bin/bash
➔ echo "What is your name?"
➔ read NAME
➔ echo "Welcome, $NAME!"

Arguments passed at runtime:

➔ #!/bin/bash
➔ echo "First argument: $1"
➔ echo "All arguments: $@"
➔ echo "Number of arguments: $#"

Run it with: `./script.sh Alice Bob`

Conditionals

```
\!/bin/bash

AGE=$1

if [ $AGE -ge 18 ]; then

  echo "You are an adult."

else

  echo "You are a minor."

fi
```

Common comparison operators:

- `-eq` equal
- `-ne` not equal
- `-gt` greater than
- `-lt` less than
- `-ge` greater than or equal
- `-le` less than or equal
- For strings: `=` and `!=`

Check if a file exists:

```
if [ -f "/etc/passwd" ]; then

  echo "File exists."

fi
```

- `-f` = is a file
- `-d` = is a directory
- `-e` = exists (either)

Loops

For loop:

```
#!/bin/bash
for i in 1 2 3 4 5; do
    echo "Number: $i"
done
```

Loop over files:

```
for file in *.txt; do
    echo "Processing $file"
done
```

While loop:

```
COUNT=1
while [ $COUNT -le 5 ]; do
echo "Count: $COUNT"
COUNT=$((COUNT + 1))
done
```

Practical example:

```
for i in {1..5}; do
   mkdir "folder_$i"
done
```

Functions

Functions let you reuse blocks of code.

```
#!/bin/bash

greet() {
    echo "Hello, $1!"
}

greet "Alice"
greet "Bob"
```

A more practical example:

```
#!/bin/bash

backup_file() {
    cp "$1" "$1.bak"
    echo "Backup created: $1.bak"
}

backup_file "config.txt"
```

Debugging Scripts

Method 1 — set -x (trace mode):
Add to the top of your script to print every command before it runs:

```
#!/bin/bash

set -x
```

Method 2 — set -e:
Stops the script immediately if any command fails (prevents silent failures):

```
set -e
```

Method 3 — set -u:
Treats unset variables as errors (catches typos in variable names):

```
set -u
```

Best practice — combine them:

```
#!/bin/bash

set -euo pipefail
```

Sample System Automation Tasks

Disk usage alert:

```
#!/bin/bash

THRESHOLD=80

USAGE=$(df / | grep / | awk '{print $5}' | sed 's/%//')

if [ $USAGE -gt $THRESHOLD ]; then

    echo "WARNING: Disk usage is at ${USAGE}%"

fi
```

Auto-backup a directory:

```
#!/bin/bash

SOURCE="/home/user/projects"

DEST="/backup/$(date +%Y-%m-%d)"

mkdir -p $DEST

cp -r $SOURCE $DEST

echo "Backup complete: $DEST"
```

Crontab (Scheduling Tasks)

What is Cron?

Cron is a background scheduler for repetitive tasks. A crontab is like your magical alarm clock. Instead of ringing a bell, it tells your computer: “Hey, run this job at this exact time, every day/week/month/etc.” **Cron** is essentially about understanding how to tell a Linux computer: "Do this specific thing, at this specific time, repeatedly."

- Cron = The actual system service (daemon) that runs in the background. It wakes up every minute to check if there is work to do. It’s like the little robot in your computer that checks the time every minute.
- Crontab = The list of instructions you give to that robot, we can say it’s a to-do list.
- CronJob = A single task or line in a crontab

Managing Your Crontab

You generally do not edit the crontab file directly with a text editor. Instead, you use the crontab command in your terminal.

- crontab -e (Edit): Opens your personal crontab file in your default text editor (usually Vim or Nano). This is how you add or modify jobs.
- crontab -l (List): Prints the contents of your currently active crontab to the screen.
- crontab -r (Remove) Warning: This deletes your entire crontab file.

The Crontab Syntax (The 5 Stars)

Every line in a crontab file follows a strict pattern consisting of 5 time fields followed by the command to run.

The format looks like this:* * * * * command_to_execute

And here is what each position represents:

Position	Field	Allowed Values
1	Minute	0-59
2	Hour	0-23
3	Day of Month	1-31
4	Month	1-12 (Or names like JAN, FEB)
5	Day of Week	0-6 (0 is Sunday, 6 is Saturday)

Tips on Cron

The power of cron comes from the special characters you can put in those fields:

- *** (Asterisk):** Means "every."

* in the Hour field means "every hour."

- **, (Comma):** Used to list multiple specific values. 1,15 in the Minute field means "at minute 1 AND minute 15."
- **- (Hyphen):** Used to define a range. 9-17 in the Hour field means "every hour from 9 AM to 5 PM."
- **/ (Slash):** Used for step values (intervals). */10 in the Minute field means "every 10 minutes."

More tips

Always use Absolute Paths: Cron runs in a very minimal environment. It does not know where your files are. **Bad:** python script.py **Good:** /usr/bin/python3 /home/user/scripts/script.py 2.

Try running the command manually in your terminal first to ensure the syntax and permissions are correct.

Use a Generator: Memorizing the syntax is good, but for complex schedules, use a tool like c It allows you to type a schedule and see a plain English translation.

How do Crontabs and Bash Scripts work together?

Think of it like this:

Bash Script = The chore itself.

Example: “Wash the dishes.” (commands inside the script)

Crontab = The schedule to do the chore.

Example: “Wash the dishes at 7 PM every day.” (when to run the script)

In short, Bash Script is the what to do, Crontab is the when to do it.

Note: You can run a Bash script any time manually, but Crontab will run it automatically at scheduled times.

The Crontab Time Code



The crontab time code works like this:

minute hour day_of_month month day_of_week command

For example: 0 6 * * * echo "Hi, Good morning!"

This command will run

0 → minute (0th minute = on the hour)

6 → hour (6 AM)

* → every day

* → every month

* → every weekday

echo "Hi Good morning!" → the task



Let’s run a test Crontab/job….

nano test_script.sh…

#!/bin/bash

echo "Hello from Cron at $(date)" >> /tmp/cron_test.log

chmod +x test_script.sh

crontab -e

* * * * * /full/path/to/test_script.sh : Use realpath filename or pwd

(sudo apt install realpath)

cat /tmp/cron_test.log

Delete cron job



Let's look at other examples

Every day at 9:02 AM: 2 9 * * *

At 2:30 PM on weekdays (Monday–Friday): 30 (minutes) 14(2 pm) * * 1-5

Every 2 hours: 0 */2 * * *

Every 5 minutes: */5 * * * *

Extra Learning Materials

Refer to Man Pages, Docs, etc

https://storm.cis.fordham.edu/~zhang/cs3130/slides/ShellAdvanced.pdf

https://www.macs.hw.ac.uk/~hwloidl/Courses/LinuxIntro/x864.html

https://www.gnu.org/software/bash/manual/bash.html

https://www.tutorialspoint.com/unix_commands/bash.htm

TutorialsPoint Bash Guide: https://www.tutorialspoint.com/unix/shell_scripting.htm

Cron expression builder: https://crontab.guru

Practice Questions…

- Write a script that prints "Hello, World!" to the terminal.
- What does `chmod +x script.sh` do and why is it necessary?
- A script is failing silently. What would you add to help debug it?
- Write a cron expression that runs a script every day at 6am.
- What's the difference between `$1` and `$@` in a bash script?

Questions…..

....

